

Anomaly Detection

1st Paper Research

2026.03.11

Background

Anomaly Detection

- 정상 데이터의 패턴을 학습한 후, 해당 패턴에서 크게 벗어나는 데이터를 Anomaly로 탐지하는 기술임
- 제조 공정, 의료 영상, 보안 등 다양한 분야에서 품질 이상이나 오류를 조기에 탐지하기 위해 사용됨

Supervised Anomaly Detection

- 정상 / 비정상 데이터와 라벨이 모두 존재하고 높은 정확도 달성 가능
- 그러나 실제 산업 환경에서는 비정상 데이터가 매우 희귀함 (Class Imbalance 문제)

One-Class (Semi-supervised) Anomaly Detection

- 정상 데이터만으로 모델 학습함
- 비정상 데이터 확보가 어려운 산업 환경에서 유용함

Background

Vision-based Anomaly Detection Methods

1. Reconstruction-based

- 정상 이미지의 구조를 학습하여 입력 이미지를 복원하고, 복원 오류(Reconstruction Error)를 이용해 이상 탐지 (Autoencoder, GAN, Diffusion)

2. Density-based

- 정상 데이터의 확률 밀도 분포를 모델링하고 낮은 확률 밀도를 가지는 샘플을 이상으로 판단 (Gaussian Mixture Model, Normalizing Flow)

3. Distance-based

- 정상 데이터의 대표 feature와의 거리를 계산하여 멀리 떨어진 샘플을 이상으로 탐지

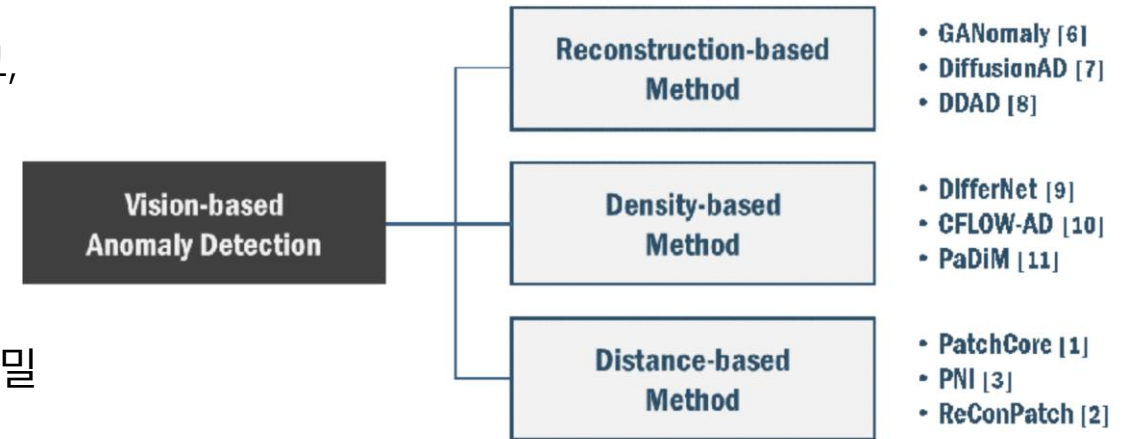
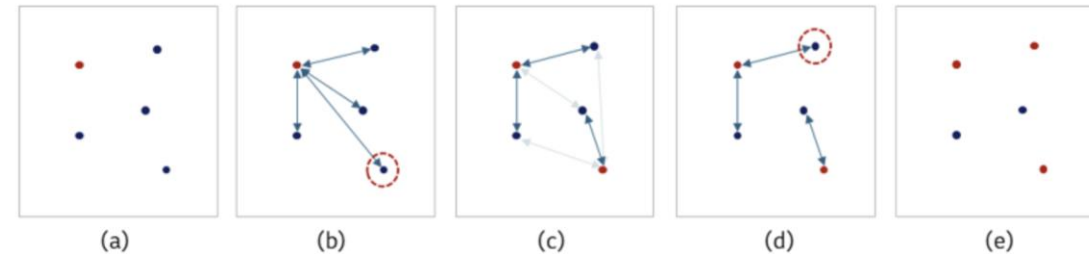


그림 2. 제품 이미지에서 이상을 감지하는 3가지 형태

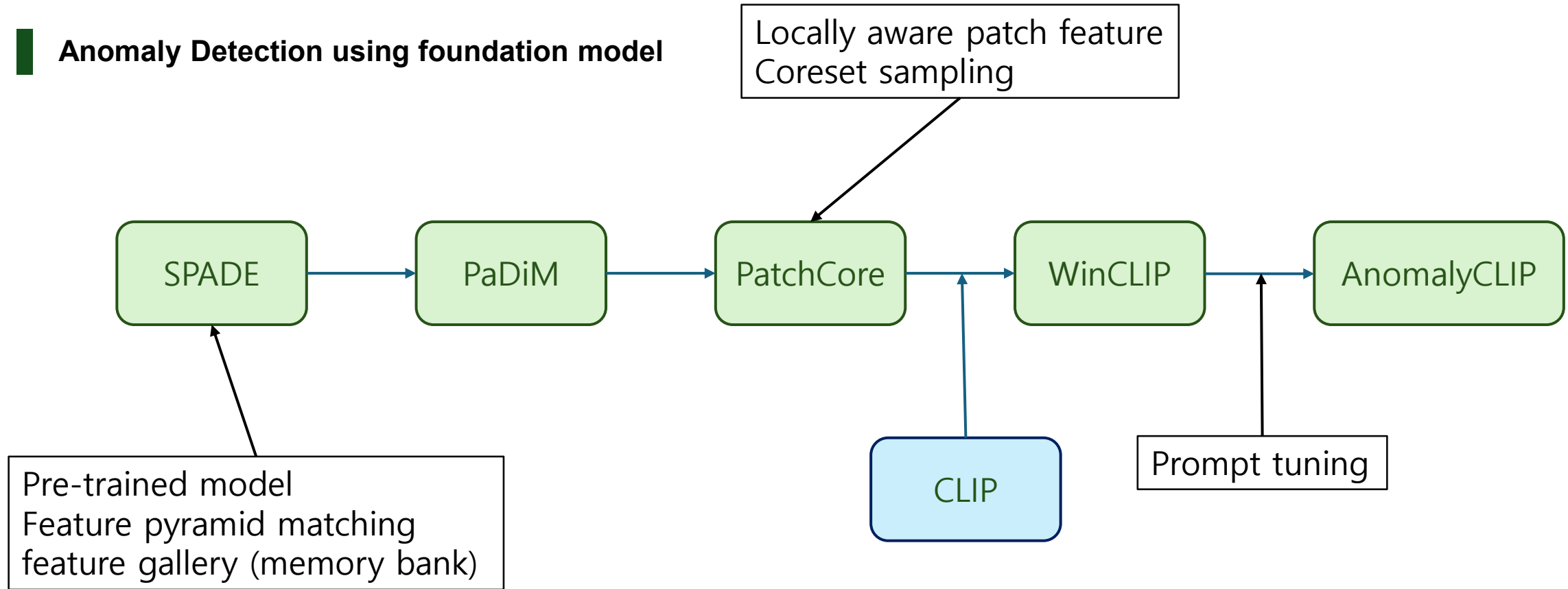


Anomaly Detection using foundation model

- PatchCore: Towards Total Recall in Industrial Anomaly Detection
- WinCLIP: Zero-/Few-Shot Anomaly Classification and Segmentation
 - CLIP: Learning Transferable Visual Models From Natural Language Supervision
 - SPADE: Semi-supervised Anomaly Detection under Distribution Mismatch
 - PaDiM: a Patch Distribution Modeling Framework for Anomaly Detection and Localization
 - ANOMALYCLIP: OBJECT-AGNOSTIC PROMPT LEARNING FOR ZERO-SHOT ANOMALY DETECTION

Outline

Anomaly Detection using foundation model



Feature extraction with Pretrained model

- 별도의 정상 이미지 학습 과정을 거치지 않음
 - 이미지넷 등의 큰 데이터셋으로 사전학습된 모델을 사용하여 feature를 추출함
 - 사전학습된 모델을 사용하여 추출한 feature는 정상 feature들끼리 모여있고 비정상 feature는 이로부터 떨어져 있을 것으로 가정함
 - 정상 이미지 feature들로부터 멀리 떨어져 있을수록 불량이라고 판단함
- kNN (k Nearest Neighbor) 이용

Feature pyramid matching

- ✓ High level layer - semantic 정보 많음
 - ✓ Low level layer - fine-grained information 많음
 - 각 layer level의 정보를 다 사용하기 위해 하나로 합침
 - 다른 layer의 feature들은 resolution이 다름
- 큰 resolution에 맞게 resize 하여 channel 방향으로 합쳐 하나의 feature로 만들

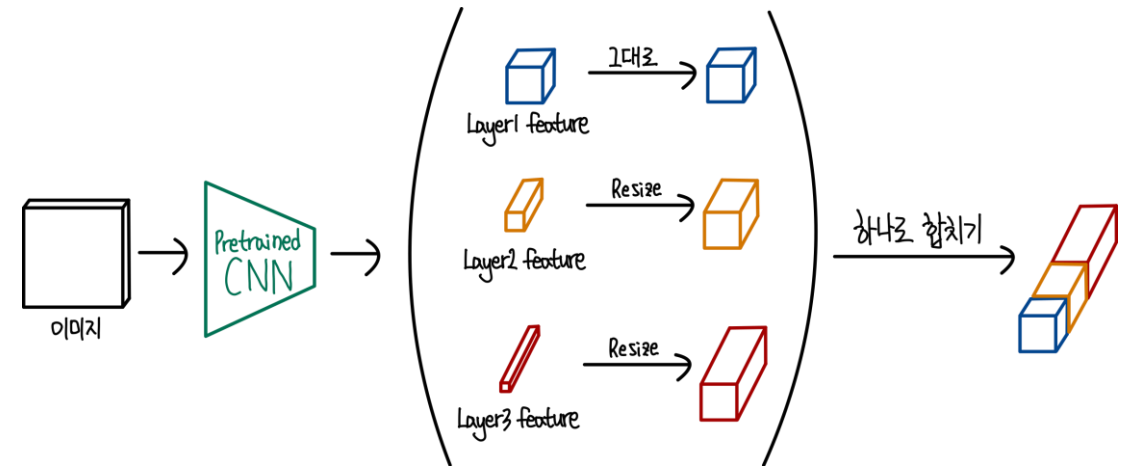
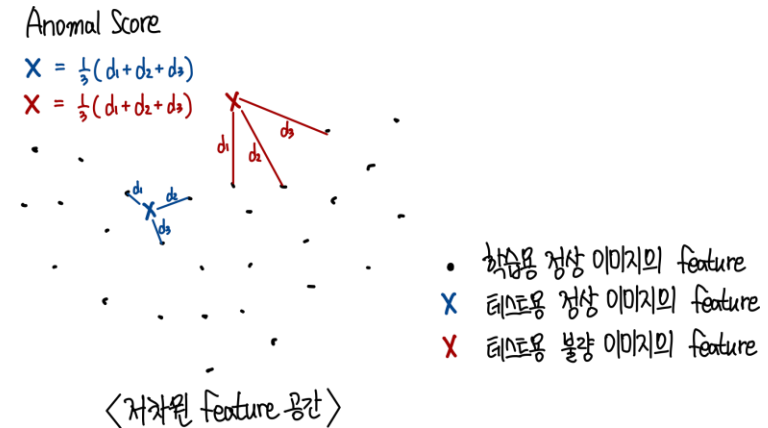
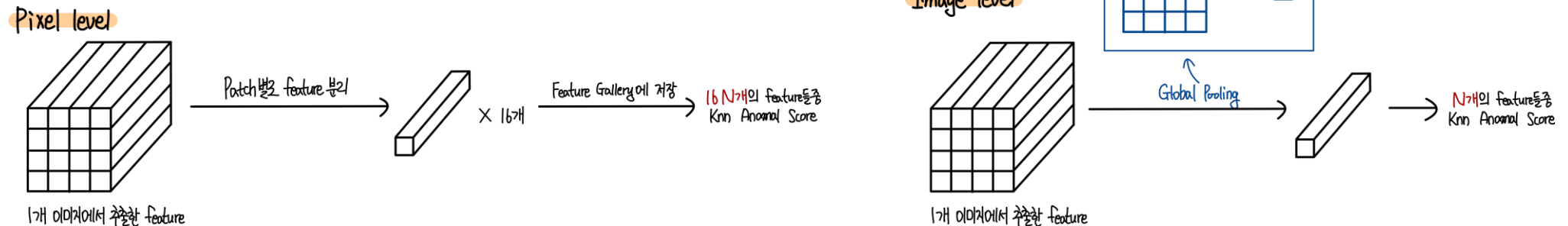


Image level

- 사전학습된 모델을 사용하여 추출된 feature를 Global pooling하여 최종 feature로 만들어 줌
- 이미지 개수만큼 feature가 나오고 knn으로 거리를 측정함

* Global Pooling 사용 이유: Image level에서 Anomaly Detection을 하려면 정보가 어느 위치에 있건 상관없음 (pixel level과 다름)



Pixel level

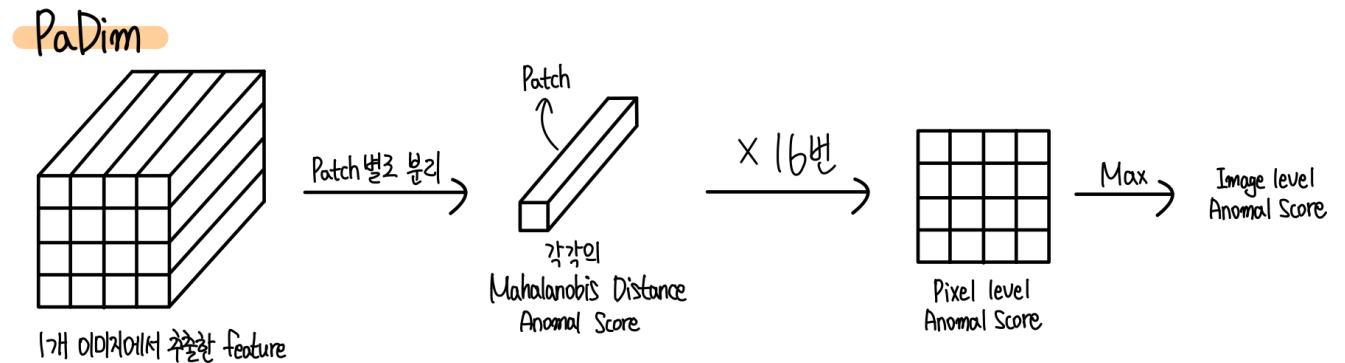
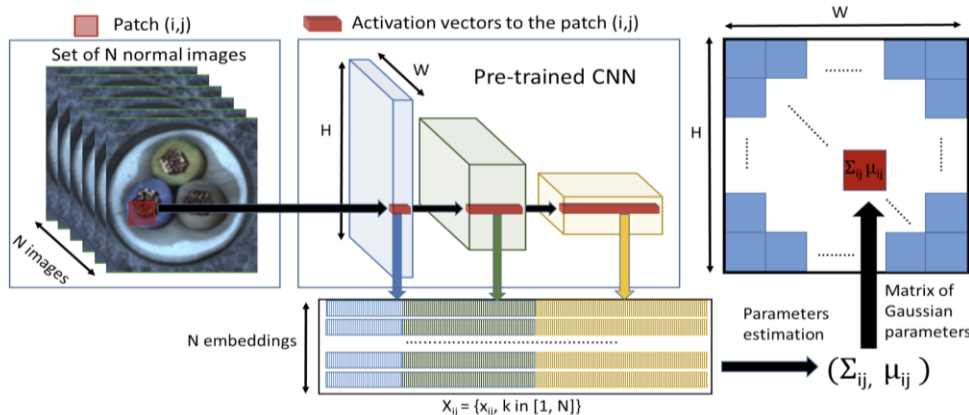
- 추출한 feature를 pooling하지 않고 모든 patch별 feature를 메모리에 저장함
- 모든 patch들의 feature를 feature gallery (memory bank)에 올려놓고 테스트 단계에서 kNN으로 anomaly score를 측정함
- Irrelevant to image alignment

Embedding extraction

- SPADE와 마찬가지로 Pretrained CNN 사용하여 feature를 추출 & multi-level concatenate

Learning normality

- 각 patch 위치마다 정상 feature distribution을 학습함
- 각 위치 feature가 multivariate Gaussian이라고 가정하고 mean과 covariance를 계산함
- 모든 위치마다 Gaussian parameter를 저장하여 map을 만듦



Inference

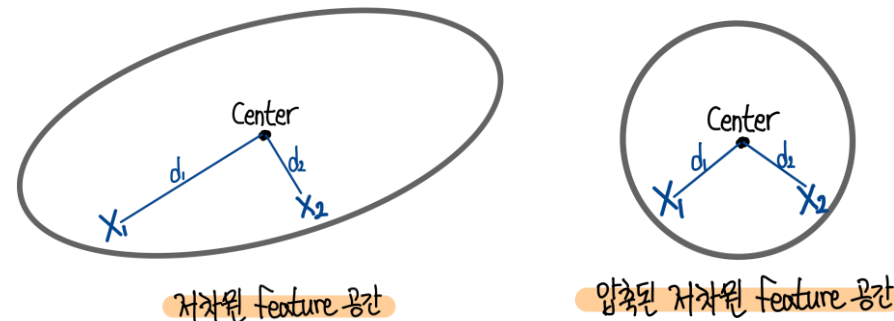
- test image가 들어오면 같은 방식으로 patch embedding x_{ij} 생성
- Mahalanobis distance 계산

$M(x_{ij}) = \sqrt{(x_{ij} - \mu_{ij})^T \Sigma_{ij}^{-1} (x_{ij} - \mu_{ij})} \rightarrow$ 각 위치에서 test patch가 정상 distribution에서 얼마나 떨어져 있는가

- anomaly map 생성 $M = (M(x_{i,j})) \rightarrow$ anomaly heatmap
- 최종 anomaly score = anomaly map의 maximum

Comparison (SPADE, PaDiM)

- SPADE와 PaDiM은 pretrained model & Feature pyramid matching으로 feature 만들고 patch level로 구분하여 feature를 선택함
- 그러나 SPADE는 모든 patch를 메모리에 올려 비교함 $\rightarrow$ Irrelevant to image alignment
- PaDiM은 나와 동일한 위치의 patch들과만 비교함 $\rightarrow$ translation, rotation에 영향 많이 받음

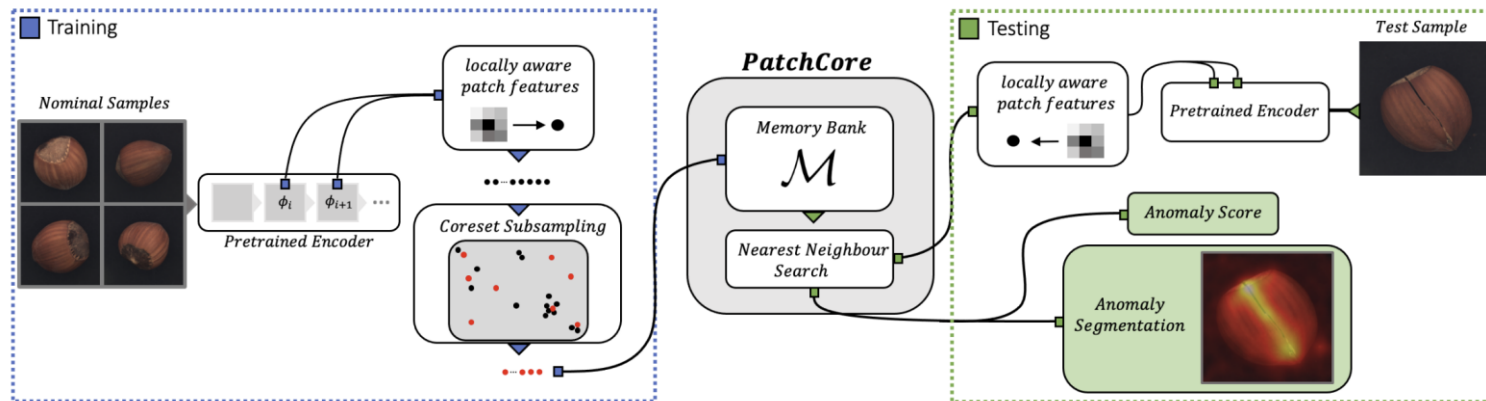


PatchCore

Structure

Training phase

- pretrained network를 사용하여 feature를 추출함
- 추출한 feature들은 locally aware patch feature로 만듦
- 그 다음 coreset sampling을 통해 메모리에 올릴 patch feature들을 선별함
- 선택한 patch feature들을 모두 메모리에 넣고 이는 test 단계에서 anomaly를 판단하는데 사용됨



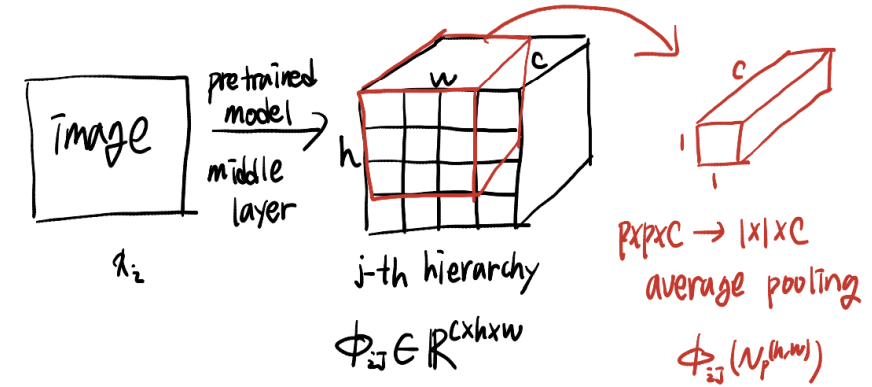
Testing phase

- Training과 마찬가지로 pretrained network로 추출한 feature를 locally aware 하게 만듦
- Memory bank의 patch feature들과 비교해 anomaly score를 측정함

PatchCore - Training phase

Mid level feature 사용

- High level feature는 너무 semantic하고 ImageNet classification에 특화됨
 - anomaly detection은 local texture / spatial pattern 정보도 중요함
- ResNet 기준 layer 2, layer 3 (mid-level feature) 사용함



Locally aware patch feature

Image i -th / hierarchy j -th

- ❖ ϕ : pretrained network
 - ❖ $\mathcal{N}_p^{(h,w)}$: (h,w) 를 중심으로 하는 $p \times p$ neighborhood
 - ❖ f_{agg} : aggregation function (average pooling)
 - 주변 patch feature들을 평균하여 local context를 반영한 patch feature 생성함
- Local texture 유지하면서 local context를 반영하여 patch feature의 표현력을 높이기 위함

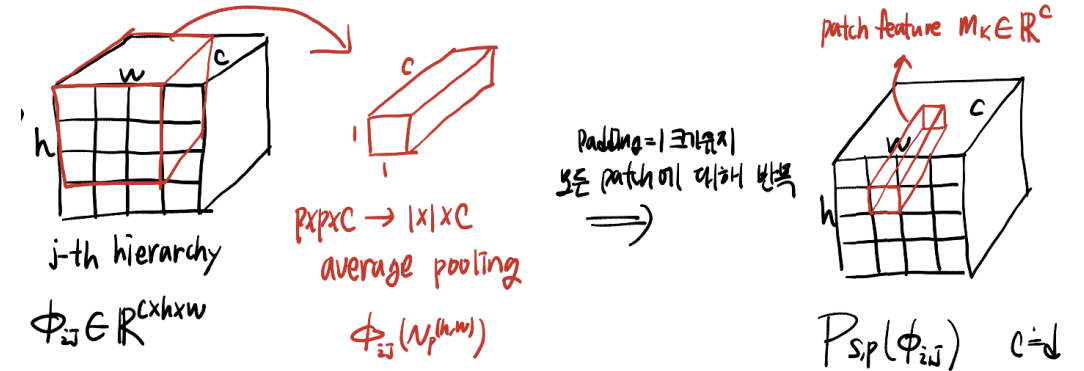
$$\mathcal{N}_p^{(h,w)} = \{(a,b) | a \in [h - \lfloor p/2 \rfloor, \dots, h + \lfloor p/2 \rfloor], b \in [w - \lfloor p/2 \rfloor, \dots, w + \lfloor p/2 \rfloor]\}, \quad (1)$$

$$\phi_{i,j}(\mathcal{N}_p^{(h,w)}) = f_{agg} \left(\{ \phi_{i,j}(a,b) | (a,b) \in \mathcal{N}_p^{(h,w)} \} \right), \quad (2)$$

PatchCore - Training phase

Patch feature collection 생성

- 모든 위치 (h,w)에 대해 locally aware feature를 계산하면 patch feature 집합이 만들어짐
- feature map의 모든 위치에 대해 patch feature 생성함



Multi-layer feature aggregation

- PatchCore는 CNN의 두 개의 intermediate layer feature를 사용함
- 두 feature map을 bilinear interpolation + concatenation 하여 multi-scale patch feature 생성함

Memory bank

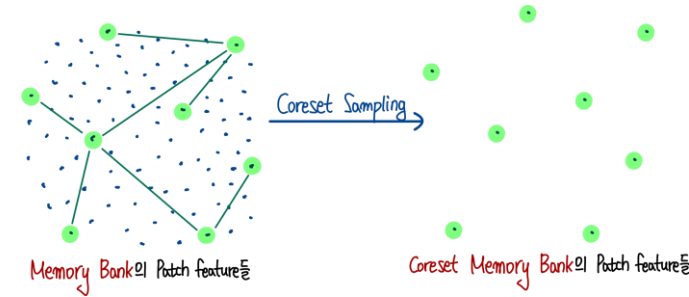
- training data의 모든 정상 이미지에 대해 patch feature들을 추출하여 memory bank M에 저장함
- 그러나 메모리 사용량 증가하고 nearest neighbor 검색 비용 증가한다는 문제가 발생함

$$\therefore \text{Memory Bank } \mathcal{M} = \bigcup_{i,j \in \mathcal{I}} P_{sp}(\phi_{ij}) = \{m_1, m_2, \dots, m_K\}$$

PatchCore - Training phase

Coreset sampling

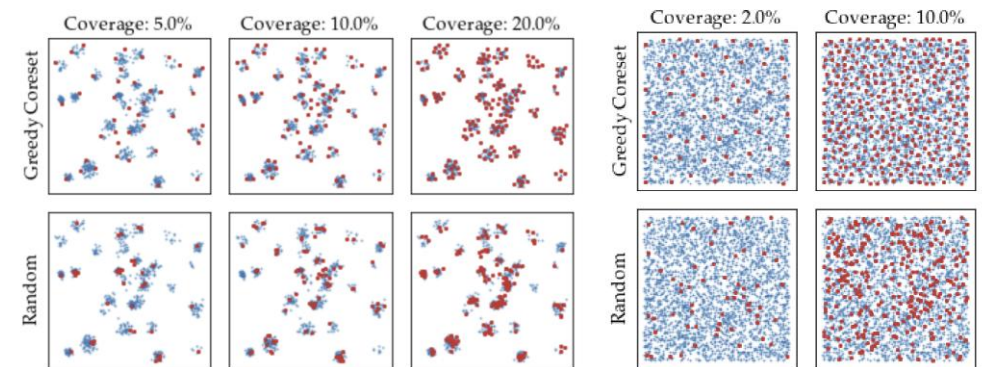
- 원래 memory bank M 을 작은 subset M_C 로 줄이되 feature space coverage는 유지함
- ✓ Coverage 목적: anomaly detection에서는 rare normal도 분포 안에 포함시켜야함, Random sampling 했을 경우 모든 영역을 커버하지 못함
- normal distribution을 완전히 덮는 방식으로 minimax objective 사용함



$$\mathcal{M}_C^* = \arg \min_{\mathcal{M}_C \subset \mathcal{M}} \max_{m \in \mathcal{M}} \min_{n \in \mathcal{M}_C} \|m - n\|_2.$$

- 모든 original feature m 에 대해 subset M_C 안에서 가장 가까운 point n 을 찾는다.
 - 전체 데이터 중 가장 멀리 떨어진 것을 본다.
 - 이 값을 최소가 되도록 M_C 를 고른다.
- 어떤 feature도 coreset에서 너무 멀어지지 않도록 함

- 해당 문제는 NP-hard 이므로 실제 계산은 greedy로 근사함
- memory bank에서 하나 선택, 기존 선택된 feature와 가장 멀리 떨어진 feature 추가, 반복하면서 diverse feature set 생성



PatchCore – Testing phase

Patch anomaly score

test 이미지 x^{test} 가 들어오면

1. patch feature를 추출하고
2. memory bank M 의 feature와 nearest neighbor 거리를 계산함

Image anomaly score

- 정상 feature와 가장 멀리 떨어진 patch의 거리가 anomaly 후보가 됨

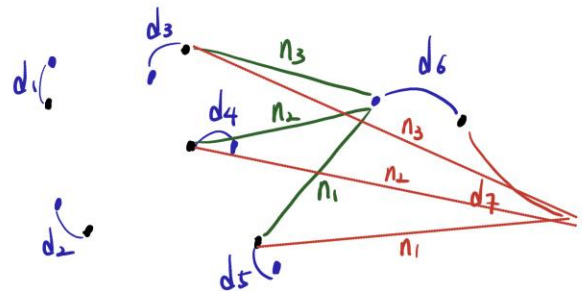
$$m^{\text{test},*}, m^* = \arg \max_{m^{\text{test}} \in \mathcal{P}(x^{\text{test}})} \arg \min_{m \in \mathcal{M}} \|m^{\text{test}} - m\|_2$$

$$s^* = \|m^{\text{test},*} - m^*\|_2.$$

$$s = \left(1 - \frac{\exp \|m^{\text{test},*} - m^*\|_2}{\sum_{m \in \mathcal{N}_b(m^*)} \exp \|m^{\text{test},*} - m\|_2} \right) \cdot s^*$$

Re-weighting

- Rare한 normal 패턴의 경우, 단순 max distance가 크기 때문에 false positive 발생함
- 주변 patch distribution을 고려하여 score를 보정함
- ❖ m^* : nearest neighbor
- ❖ $\mathcal{N}_b(m^*)$: memory bank에서 m^* 주변 patch들



⑥은 normal

⑦은 anomalous

re-weighting ($b=s$)

$$S = \left(1 - \frac{\exp(d_6) \text{ weight } \downarrow}{\exp(n_1) + \exp(n_2) + \exp(n_3)} \right) \cdot d_6 \Rightarrow \text{score 작아짐}$$

$$S = \left(1 - \frac{\exp(d_7) \text{ weight } \uparrow}{\exp(n_1) + \exp(n_2) + \exp(n_3)} \right) \cdot d_7 \Rightarrow \text{score 커짐}$$

PatchCore - Experiment

mvTec

Image-level anomaly detection

- PaDiM: 2.1% / PatchCore-25%: 0.9% → 57% error 감소
- 1725 test 중 42개만 Misclassification
- 15 class 중 5 class는 완벽하게 탐지함
- memory bank를 줄여도 성능이 유지됨

Anomaly Segmentation

- PatchCore가 SOTA

Other benchmarks

mSTC & MTD 두 데이터 셋에서 PatchCore가 가장 우수한 성능을 보임

Table 6. Anomaly Segmentation on mSTC [32, 52] and anomaly detection on MTD [26] compared to results reported in [42].

mSTC	CAVGA-R _u [52]	SPADE [10]	PaDiM [14]	PatchCore-10
Pixelwise AUROC [%]	85	89.9	91.2	91.8
MTD	GANomaly [2]	1-NN [35]	DifferNet [42]	PatchCore-10
AUROC [%]	76.6	80.0	97.7	97.9

Method	SPADE [10]	PatchSVDD [56]	DifferNet [42]	PaDiM [14]	Mah.AD [40]	PaDiM* [14]	PatchCore-25%	PatchCore-10%	PatchCore-1%
AUROC ↑	85.5	92.1	94.9	95.3	95.8	97.9	99.1	99.0	99.0
Error ↓	14.5	7.9	5.1	4.7	4.2	2.1	0.9	1.0	1.0
Misclassifications ↓	-	-	-	-	-	-	42	47	49

Table 2. Anomaly Segmentation Performance (pixelwise AUROC) on MVTec AD [5].

Method	AE _{SSIM} [5]	γ -VAE + grad. [15]	CAVGA-R _w [52]	PatchSVDD [56]	SPADE [10]	PaDiM [14]	PatchCore-25%	PatchCore-10%	PatchCore-1%
AUROC ↑	87	88.8	89	95.7	96.0	97.5	98.1	98.1	98.0
Error ↓	13	11.2	11	4.3	4.0	2.5	1.9	1.9	2.0

PatchCore - Experiment

Locally aware patch-features

- neighborhood size p 를 바꿔 실험함
 - ✓ 너무 작으면 → local 정보만 봄
 - ✓ 너무 크면 → global 정보 과다 + anomaly localization 약화
 - local 정보와 global context 사이의 균형이 필요
- $p = 3$ 이 가장 좋은 성능을 달성함

Feature hierarchies

- Layer 얇으면 local detail 많음 / 깊으면 global context 많고 ImageNet bias
- Level 2가 최적이고 3을 같이 쓰면 더 좋음

Importance of Coreset subsampling

- greedy coreset selection이 가장 좋은 성능을 보임
- coreset을 사용하면 memory bank를 최대 100배 줄이면서도 성능을 유지할 수 있음

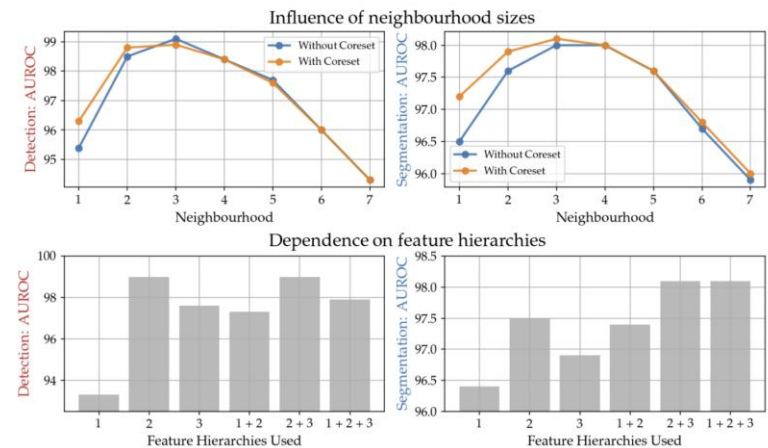


Figure 4. Local awareness and network feature depths vs. detection performance. PRO score results in the supplementary.

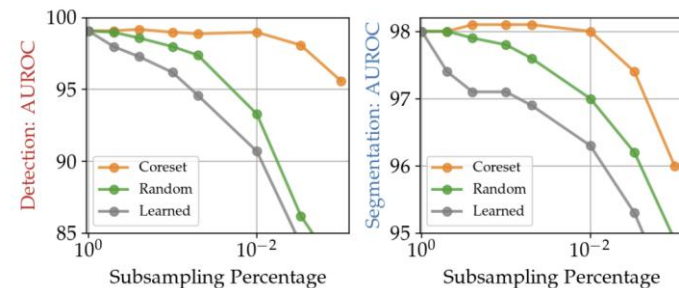


Figure 5. Performance retention for different subsamplers, results for PRO score in the supplementary.

Introduction

대부분의 연구는 정상 데이터만 사용하는 one-class / unsupervised anomaly detection에 집중함

Limitation of Existing Methods

- 기존 방법들은 각 검사 작업마다 별도의 모델을 학습해야 함
- 이는 다양한 제품과 결함이 존재하는 long-tail 문제에서 확장성이 낮음
- 특히 few-normal-shot 환경에서는 성능 향상이 여전히 어려움

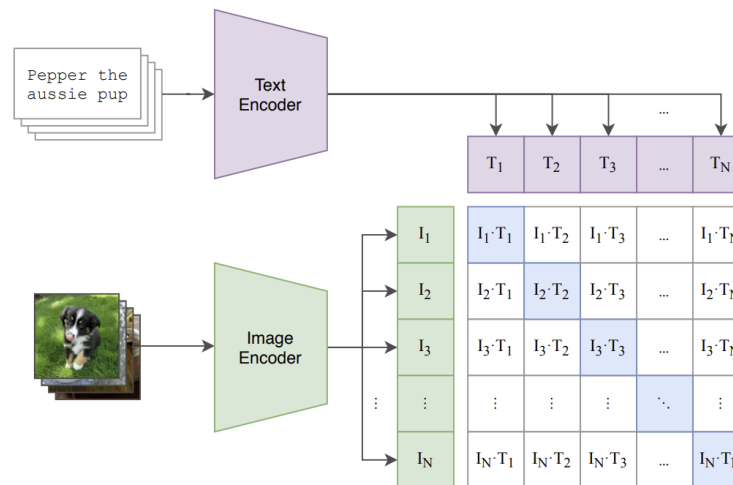
Opportunity: Vision-Language Models

- CLIP 같은 Vision-Language Model은 이미지와 텍스트의 공통 표현 공간을 학습함
- 텍스트 프롬프트를 이용하면 zero-shot / few-shot 전이 학습이 가능함
- 그러나 기존 CLIP은 global embedding 중심 학습이라 pixel-level anomaly segmentation에는 한계

Contrastive pretraining

- N개의 (이미지, 텍스트) 쌍의 배치에 주어졌을 때, CLIP은 $N \times N$ 개의 가능한 (이미지, 텍스트) 쌍을 예측하도록 학습됨
- N개의 실제 쌍의 이미지 임베딩과 텍스트 임베딩의 코사인 유사도를 최대화하고 $N^2 - N$ 개의 잘못된 쌍의 코사인 유사도는 최소화하도록 이미지 인코더와 텍스트 인코더를 같이 학습함으로써 multi-modal 임베딩 공간을 학습함
- ✓ Image Encoder: ResNet, ViT
- ✓ Text Encoder: Transformer

(1) Contrastive pre-training



```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

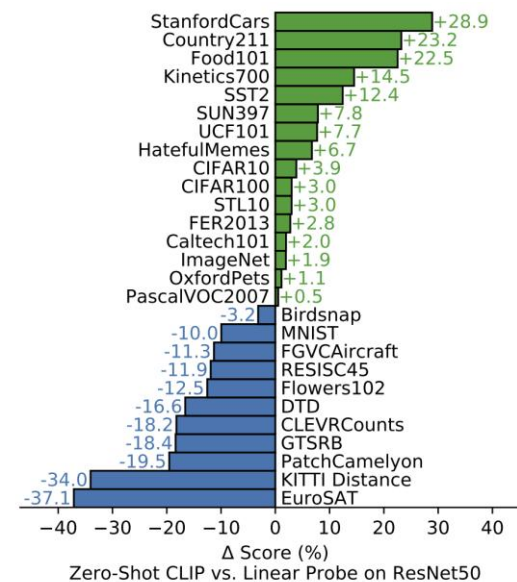
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

Zero-shot 성능

Comparison with supervised classifier

- 27개 데이터셋 평가 결과 CLIP이 16 / 27 dataset에서 baseline보다 성능이 좋음
- CLIP이 language supervision을 통해 다양한 object concept을 이미 학습했기 때문에 object recognition / semantic classification 성능이 좋음
- specialized task에서는 성능이 떨어짐



Comparison with Few-shot

- Zero-shot CLIP은 4-shot linear classifier 성능과 거의 동일함
 - 16-shot classifier와도 거의 비슷함
 - CLIP은 text prompt로 concept을 직접 전달함
- language가 concept supervision 역할

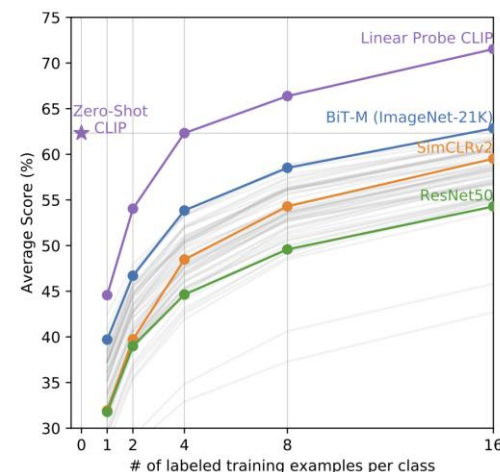


Figure 6. Zero-shot CLIP outperforms few-shot linear probes.

Introduction

Proposed Idea (WinCLIP)

- Window-based CLIP (WinCLIP): multi-scale window feature를 이용해 language-aligned spatial feature 추출
- WinCLIP+: few-normal-shot setting에서 정상 이미지 reference 정보 추가 활용
- 추가 학습 없이(zero-shot) 다양한 anomaly detection task에 적용 가능

Main Contributions

- Prompt ensemble로 zero-shot anomaly classification 성능 향상
- WinCLIP: multi-scale feature 기반 language-guided anomaly segmentation
- WinCLIP+: 정상 reference 이미지를 활용한 few-normal-shot anomaly detection
- MVTec-AD / VisA benchmark에서 SOTA 성능 달성

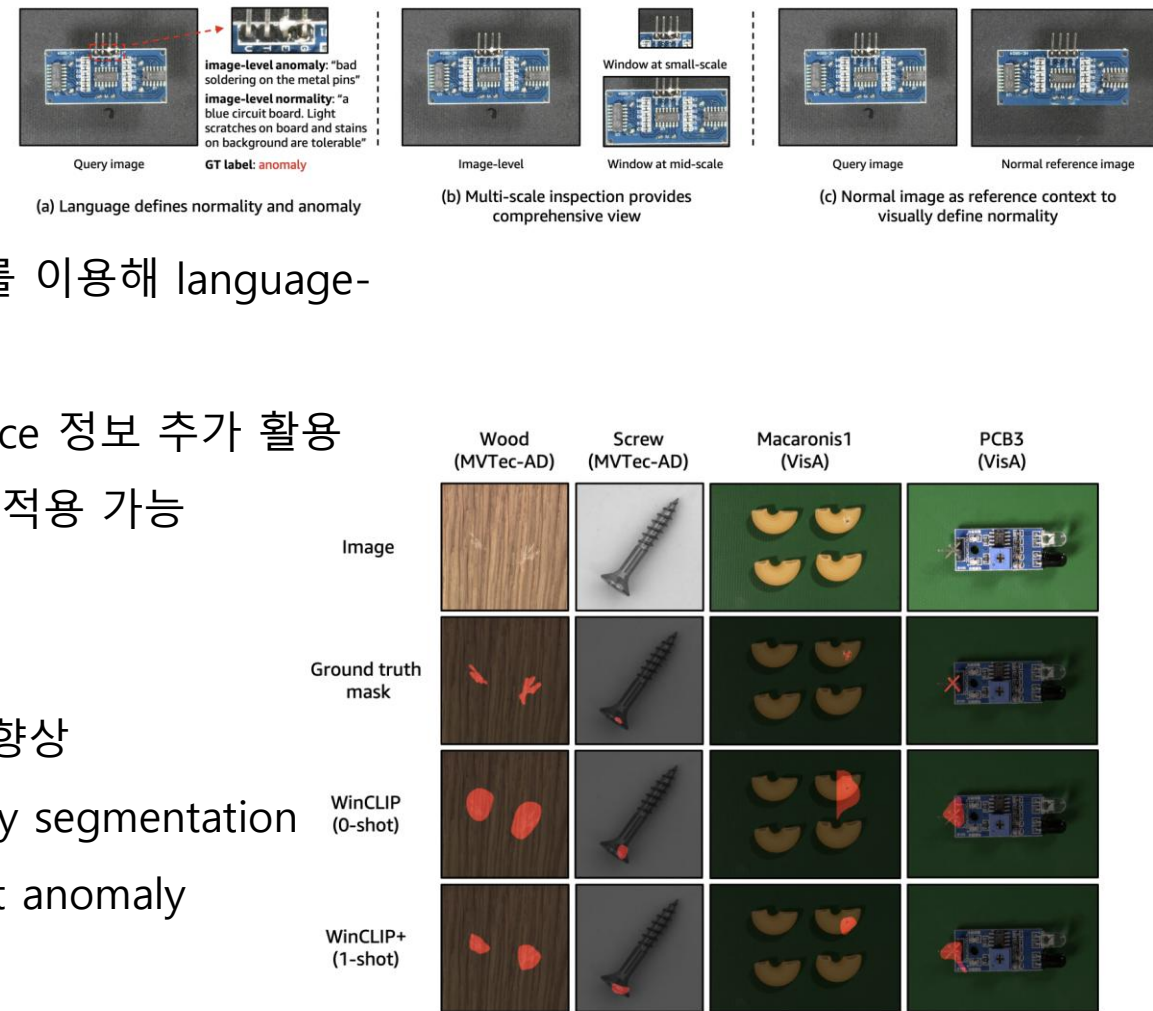


Figure 1. Language guided zero-/one-shot¹ anomaly segmentation from WinCLIP/WinCLIP+. Best viewed in color and zoom in.

WinCLIP - Language-driven zero-shot AC

Prompt $s -$: normal [object], $s +$: anomalous [object]

Two-class design $\langle f(x), g(s+) \rangle, \langle f(x), g(s-) \rangle \rightarrow \text{softmax 계산}$

One-class design $-\langle f(x), g(s-) \rangle \rightarrow \text{anomaly score (normal과 멀수록 anomaly)}$

Compositional Prompt Ensemble

State words와 text templates를 조합하여 prompt를 생성함

flawless
perfect
without defect

a photo of a [c]
a photo of a [c] for visual inspection
a cropped photo of a [c]

normal 문장 수: $3 \times 3 = 9 \rightarrow \text{mean}$

anomalous 문장 수: $3 \times 3 = 9 \rightarrow \text{mean}$

damaged
with crack
with scratch

$c \in \{c_1, c_2, \dots, c_k\}$

각 object class에 대해 prototype vector 2개

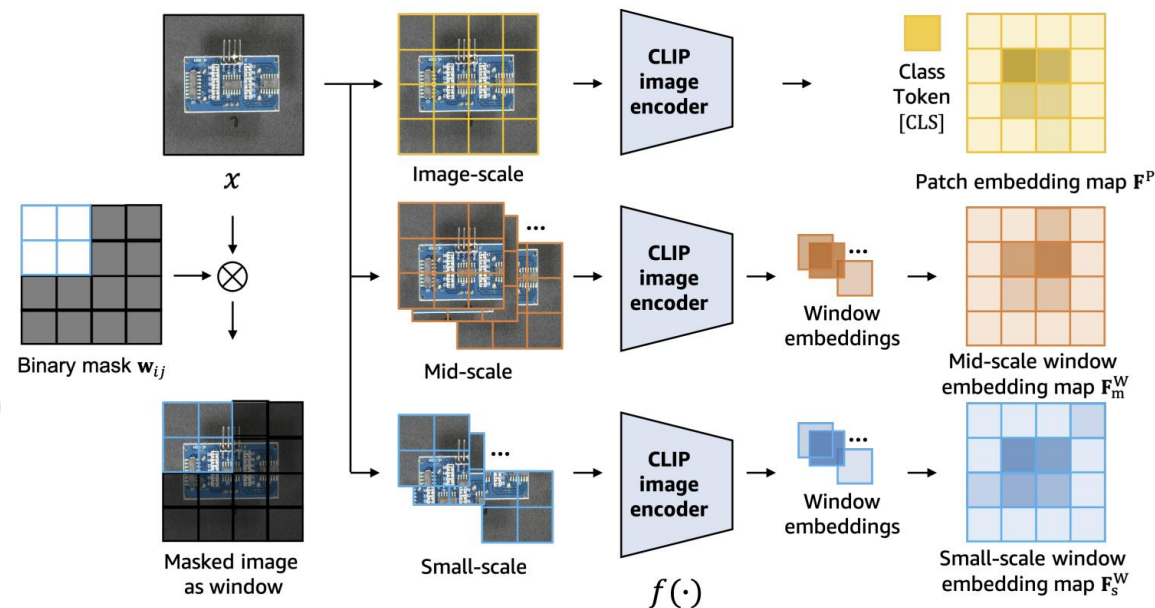
총 2k개

WinCLIP for zero-shot AS

Window-based feature extraction

CLIP은 원래 global embedding 중심 모델이기 때문에 pixel-level anomaly segmentation에 직접 사용하기 어려움

- 이미지에 sliding window를 적용
 - 각 window 영역을 CLIP encoder에 입력 $\mathbf{F}_{ij}^W := f(\mathbf{x} \odot \mathbf{w}_{ij})$
- local feature를 추출하여 dense feature map을 생성함



Window anomaly scoring

- 각 window feature와 text embedding 사이의 similarity를 계산하여 window-level anomaly score를 계산함
- Harmonic averaging 이용함 → 작은 값에 더 민감함
- 윈도우 중 하나라도 normal 값에 가까우면 전체 score가 작아짐 → false positive 감소

$$\bar{M}_{0,ij}^W := \left(\frac{1}{\sum_{u,v} (\mathbf{w}_{uv})_{ij}} \sum_{u,v} \frac{(\mathbf{w}_{uv})_{ij}}{M_{0,uv}^W} \right)^{-1}$$

$w_{i,j} \rightarrow \text{Image Encoder} \rightarrow \text{CLS}_{w_{i,j}}$
 Prototype vector

similarity 계산 → $M_{i,j}$ anomaly score 계산
 → 윈도우별로 harmonic averaging & multi-scale 처리

WinCLIP+ with few-normal-shots

Motivation

텍스트만으로 정의하기 어려운 anomaly가 존재하기 때문에 few normal reference images를 활용하는 방법을 제안함

Reference association

정상 이미지 k장 $D = x_1, x_2, \dots, x_k$

Feature 세가지 $\left. \begin{array}{l} F_s^w \text{ small-scale} \\ F_m^w \text{ mid-scale} \end{array} \right\}$ sliding window 적용, 각 window에서 CLS embedding 생성 $\rightarrow F_{s,k}^w \in \mathbb{R}^{h \times w \times d}$

F^p penultimate patch $\rightarrow$ projection 전에 나오는 마지막 transformer block의 patch tokens 저장

Reference memory scale별로 $R_s^w, R_m^w, R^p \rightarrow$ 정상일때 가능한 모든 local 패턴들의 모음

Anomaly Score

$$\mathbf{M}_{ij} := \min_{r \in \mathbf{R}} \frac{1}{2} (1 - \langle \mathbf{F}_{ij}, r \rangle)$$

Query 이미지의 한 위치가 'r' 정상 패턴들중 하나와 매우 비슷하면 정상, 어떤 정상 패턴과도 다르면 이상

* 평균내면 정상의 다양성이 사라지고 작은 구조차이 구분 못하므로 memory bank를 씀

$$\mathbf{M}^w := \frac{1}{3} (\mathbf{M}^p + \mathbf{M}_s^w + \mathbf{M}_m^w) \quad \text{ascore}_w(\mathbf{x}) := \frac{1}{2} \left(\text{ascore}_0(f(\mathbf{x})) + \max_{ij} \mathbf{M}_{ij}^w \right)$$

Experiment

Zero-shot Classification

- WinCLIP은 기존 CLIP 기반 anomaly detection (CLIP-AC) 대비 큰 성능 향상을 보임
- 단순 CLIP prompt 방식보다 WinCLIP이 훨씬 효과적임

Few-normal-shot Classification

- WinCLIP+는 기존 SOTA 방법들보다 높은 성능을 달성함
- PatchCore 대비 +9.7% (MVTec-AD 1-shot), +5.3% (VisA 1-shot)

Reference image 효과

- few-normal-shot을 사용할 경우 WinCLIP+ 1 / 2 / 4-shot 모두 성능 증가함
- 정상 reference 이미지가 anomaly detection 성능 향상에 기여함

Anomaly Classification		MVTec-AD			VisA		
Setup	Method	AUROC	AUPR	F_1 -max	AUROC	AUPR	F_1 -max
0-shot	CLIP-AC [27]	74.0±0.0	89.1±0.0	88.5±0.0	59.3±0.0	67.0±0.0	74.4±0.0
	+ Prompt ens. [27]	74.1±0.0	89.5±0.0	87.8±0.0	58.2±0.0	66.4±0.0	74.0±0.0
	WinCLIP (ours)	91.8±0.0	96.5±0.0	92.9±0.0	78.1±0.0	81.2±0.0	79.0±0.0
1-shot	SPADE [7]	81.0±2.0	90.6±0.8	90.3±0.8	79.5±4.0	82.0±3.3	80.7±1.9
	PaDiM [8]	76.6±3.1	88.1±1.7	88.2±1.1	62.8±5.4	68.3±4.0	75.3±1.2
	PatchCore [31]	83.4±3.0	92.2±1.5	90.5±1.5	79.9±2.9	82.8±2.3	81.7±1.6
	WinCLIP+ (ours)	93.1±2.0	96.5±0.9	93.7±1.1	83.8±4.0	85.1±4.0	83.1±1.7
2-shot	SPADE [7]	82.9±2.6	91.7±1.2	91.1±1.0	80.7±5.0	82.3±4.3	81.7±2.5
	PaDiM [8]	78.9±3.1	89.3±1.7	89.2±1.1	67.4±5.1	71.6±3.8	75.7±1.8
	PatchCore [31]	86.3±3.3	93.8±1.7	92.0±1.5	81.6±4.0	84.8±3.2	82.5±1.8
	WinCLIP+ (ours)	94.4±1.3	97.0±0.7	94.4±0.8	84.6±2.4	85.8±2.7	83.0±1.4
4-shot	SPADE [7]	84.8±2.5	92.5±1.2	91.5±0.9	81.7±3.4	83.4±2.7	82.1±2.1
	PaDiM [8]	80.4±2.5	90.5±1.6	90.2±1.2	72.8±2.9	75.6±2.2	78.0±1.2
	PatchCore [31]	88.8±2.6	94.5±1.5	92.6±1.6	85.3±2.1	87.5±2.1	84.3±1.3
	WinCLIP+ (ours)	95.2±1.3	97.3±0.6	94.7±0.8	87.3±1.8	88.8±1.8	84.2±1.6

Table 1. Comparison of anomaly classification (AC) performance on MVTec-AD and VisA benchmarks. We report the mean and standard deviation over 5 random seeds for each measurement. Bold indicates the best performance.

Experiment

Zero-shot segmentation

기존에는 zero-shot anomaly segmentation 연구가 거의 없기 때문에 CLIP 기반 segmentation 방법들과 비교함

- WinCLIP이 두 방법(Trans-MM, MaskCLIP)보다 큰 성능 향상
- 기존 CLIP adaptation보다 WinCLIP 구조가 훨씬 효과적

Few-normal-shot segmentation

- SPADE, PaDiM, PatchCore와 비교함
- WinCLIP+가 모든 metric에서 최고 성능 달성함

Reference image 효과

- few-normal-shot을 사용할수록 성능이 증가함
- 정상 reference 이미지가 segmentation 성능 향상에 기여함

Anomaly Segmentation		MVTec-AD			VisA		
Setup	Method	pAUROC	PRO	F_1 -max	pAUROC	PRO	F_1 -max
0-shot	Trans-MM [5]	57.5±0.0	21.9±0.0	12.1±0.0	49.4±0.0	10.2±0.0	3.1±0.0
	MaskCLIP [57]	63.7±0.0	40.5±0.0	18.5±0.0	60.9±0.0	27.3±0.0	7.3±0.0
	WinCLIP (ours)	85.1±0.0	64.6±0.0	31.7±0.0	79.6±0.0	56.8±0.0	14.8±0.0
1-shot	SPADE [7]	91.2±0.4	83.9±0.7	42.4±1.0	95.6±0.4	84.1±1.6	35.5±2.2
	PaDiM [8]	89.3±0.9	73.3±2.0	40.2±2.1	89.9±0.8	64.3±2.4	17.4±1.7
	PatchCore [31]	92.0±1.0	79.7±2.0	50.4±2.1	95.4±0.6	80.5±2.5	38.0±1.9
	WinCLIP+ (ours)	95.2±0.5	87.1±1.2	55.9±2.7	96.4±0.4	85.1±2.1	41.3±2.3
2-shot	SPADE [7]	92.0±0.3	85.7±0.7	44.5±1.0	96.2±0.4	85.7±1.1	40.5±3.7
	PaDiM [8]	91.3±0.7	78.2±1.8	43.7±1.5	92.0±0.7	70.1±2.6	21.1±2.4
	PatchCore [31]	93.3±0.6	82.3±1.3	53.0±1.7	96.1±0.5	82.6±2.3	41.0±3.9
	WinCLIP+ (ours)	96.0±0.3	88.4±0.9	58.4±1.7	96.8±0.3	86.2±1.4	43.5±3.3
4-shot	SPADE [7]	92.7±0.3	87.0±0.5	46.2±1.3	96.6±0.3	87.3±0.8	43.6±3.6
	PaDiM [8]	92.6±0.7	81.3±1.9	46.1±1.8	93.2±0.5	72.6±1.9	24.6±1.8
	PatchCore [31]	94.3±0.5	84.3±1.6	55.0±1.9	96.8±0.3	84.9±1.4	43.9±3.1
	WinCLIP+ (ours)	96.2±0.3	89.0±0.8	59.5±1.8	97.2±0.2	87.6±0.9	47.0±3.0

Table 4. Comparison of anomaly segmentation (AS) performance on MVTec-AD and VisA benchmarks. We report the mean and standard deviation over 5 random seeds for each measurement. Bold indicates the best performance.

Experiment

Comparison with many-shot methods

- 4-shot WinCLIP+ $\approx$ full-shot CutPaste
- 0-shot WinCLIP > 기존 few-shot 방법 (DifferNet, TDG)
- 매우 적은 정상 이미지만으로 many-shot 방법들과 경쟁 가능한 성능 달성

WinCLIP Component Ablation

- Two-class prompt가 매우 중요함
- State ensemble이 가장 큰 성능 향상 요인임
- Prompt ensemble + multi-crop은 추가적인 성능 개선 효과가 있음

Methods	Setup	AC	AS
WinCLIP (ours)	0-shot	91.8	85.1
WinCLIP+ (ours)	1-shot	93.1	95.2
WinCLIP+ (ours)	4-shot	95.2	96.2
DifferNet [32]	16-shot	87.3	-
TDG [39]	10-shot	78.0	-
RegAD-L [14]	2-shot	81.5	93.3
RegAD [14]	4 + agg.	88.2	95.8
MKD [35]	full-shot	87.7	90.7
P-SVDD [49]	full-shot	92.1	95.7
CutPaste [20]	full-shot	95.2	96.0
PatchCore [31]	full-shot	99.6	98.2

Table 2. Comparison with existing many-shot ACS methods in AUROC (or pixel-) on MVTec-AD.

Method	AUROC	AUPR	F_1 -max
(a) One-class	34.2	68.9	83.5
Two-class	74.0	89.1	88.5
(b) + State ens.	89.8	95.6	92.2
(c) + Prompt ens.	90.8	96.1	92.5
(d) + Multi-crop	91.8	96.5	92.9

Table 3. Comparison of AC performance on MVTec-AD across WinCLIP ablations in AC (Section 4.1).

Experiment

WinCLIP+ (AC) Ablations

- 언어 score와 visual reference score를 결합하는 것이 가장 좋은 성능을 보임
- language + visual reference 정보가 서로 보완적임

$$\text{ascore}_w(\mathbf{x}) := \frac{1}{2} \left(\text{ascore}_0(f(\mathbf{x})) + \max_{ij} \mathbf{M}_{ij}^w \right)$$

WinCLIP+ (AS) Ablations

- 세 가지 feature scale을 비교했을때 global만 사용하는 것보다 multi-scale을 결합 하는게 성능 향상시킴
- Small-scale은 매우 local texture를 반영하므로 noise sensitivity가 높음 $\rightarrow$ shot이 많을 때는 global + mid-scale만으로도 충분히 anomaly detection이 되는데 small-scale이 추가되면 false positive 증가해서 성능이 약간 하락하는 것으로 보임

WinCLIP+ (AC)		# shots (AUROC)			
$\max \mathbf{M}^w$	ascore_0	1	2	4	8
✓	✗	87.9	91.0	<u>92.6</u>	<u>94.5</u>
✗	✓	<u>91.8</u>	<u>91.8</u>	91.8	91.8
✓	✓	93.1	94.4	95.2	96.3

Table 5. k -shot AC ablations: MVTec-AD. Bold/underline indicate the best/runner-up.

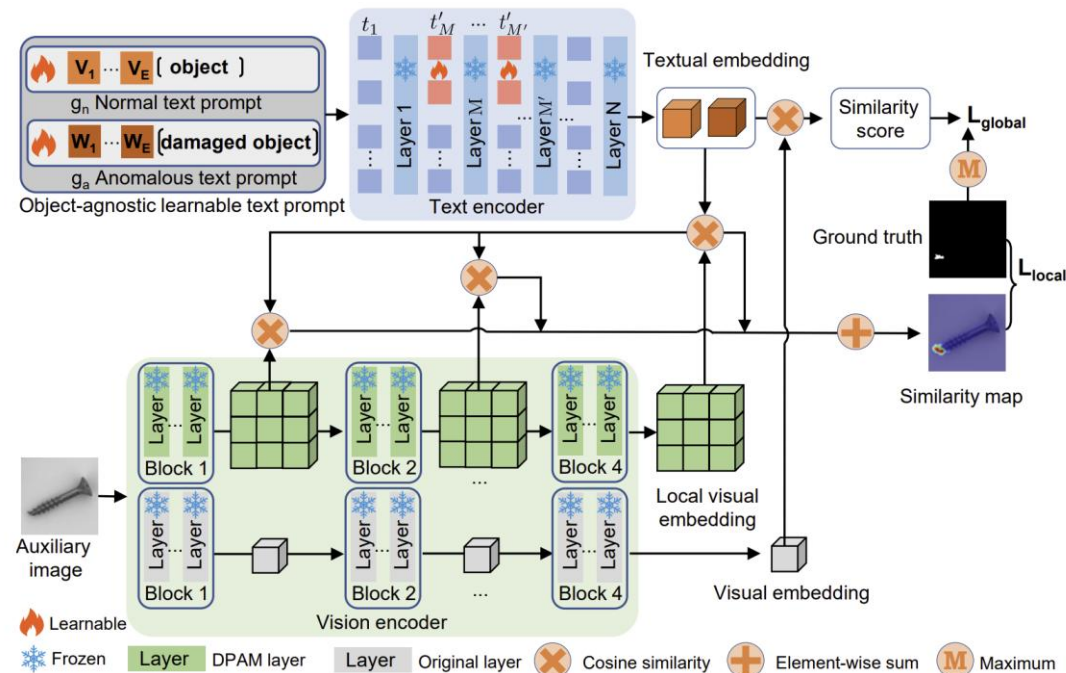
WinCLIP+ (AS)			# shots (pAUROC)			
$\mathbf{M}^p$	$\mathbf{M}_m^w$	$\mathbf{M}_s^w$	1	2	4	8
✓	✗	✗	94.5	94.8	95.4	95.8
✓	✓	✗	<u>95.1</u>	<u>95.7</u>	96.3	96.6
✓	✓	✓	95.2	96.0	<u>96.2</u>	<u>96.5</u>

Table 6. k -shot AS ablations: MVTec-AD. Bold/underline indicate the best/runner-up.

AnomalyCLIP

Overview

- Industrial AD 데이터셋을 활용한 지도 학습으로 사전학습된 CLIP을 Zero-Shot Anomaly Detection(ZSAD)에 Adapting하는 방법론
- ✓ Text Encoder의 일부 파라미터를 Prompt Tuning을 통해 학습
- ✓ Image Encoder의 파라미터는 Freeze한 상태로 V-V Attention의 연산 결과를 활용해 Pixel-Level AD를 수행함
- 객체에 상관없이 이미지 내 일반적인 정상/비정상 조건을 포착하는 Object-Agnostic Text Prompt를 Glocal Context Optimization을 통해 학습함
- Industrial 및 Medical 도메인에 해당하는 17개의 데이터셋에서 우수한 Image-/Pixel-Level AD 성능을 보임



AnomalyCLIP

Property of CLIP

- 이미지 내 Normal / Anomaly 구분하는 것보다 객체의 Class semantic을 align하도록 학습됨

Object-Agnostic Text Prompt

- CLIP에서 사용하는 Text Prompt Template는 Normal / Abnormal Semantic보단 Object Semantic에 집중함
- 이상치의 패턴은 매우 다양하기 때문에 모든 유형을 manually 반영하는 것은 현실적으로 불가능함
→ 학습 가능한 Text Prompt Template을 통한 Prompt Tuning 적용
- Cls를 [object]로 대체하여 class semantic을 제거함
- 데이터셋의 Object나 이상치 유형에 대한 지식 없이도 다양한 도메인에 적용 가능함
- WinCLIP과 같이 막대한 비용을 요구하는 Manually Defined Text Template을 사용하지 않아도 됨

$$g_n = [V_1][V_2] \dots [V_E][cls]$$

$$g_a = [W_1][W_2] \dots [W_E][damaged][cls],$$

$$g_n = [V_1][V_2] \dots [V_E][object]$$

$$g_a = [W_1][W_2] \dots [W_E][damaged][object].$$

ELLab

